

Hill Climbing Algorithm

Matz Johansson Bergström
matz.johansson@gmail.com

April 24 2026

Summary

The purpose of this project is to investigate the effectiveness and performance characteristics of a stochastic optimization approach, specifically a Hill Climbing Algorithm (HCA), for approximating raster images using a set of primitive geometric elements (colored discs right now). While the visual output provides an intuitive measure of convergence, the primary focus is not artistic reproduction but computational efficiency and scalability.

This work explores how far a simple, local-search-based method can be pushed when implemented in a high-performance computing context. Two implementations are considered: a CPU-based version in C++ and a parallel GPU-based version using CUDA. The project evaluates the trade-offs between these architectures in terms of convergence speed, solution quality, and resource utilization.

Ultimately, the project aims to answer two questions:

1. To what extent can a simple hill climbing strategy approximate complex visual data when scaled computationally?
2. What performance gains and limitations arise when transitioning from a CPU-bound implementation to a GPU-accelerated one?

1 Introduction

Approximating images using simple primitives is a useful test case for studying optimization under constrained representations. In this project, a raster image is reconstructed using a set of colored discs, optimized through a Hill Climbing Algorithm (HCA). The problem is deliberately simple in formulation but computationally demanding, making it suitable for exploring performance-oriented implementation strategies.

The work focuses on comparing a sequential CPU implementation in C++ with a parallel GPU implementation using CUDA. Rather than emphasizing visual fidelity alone, the project treats image approximation as a proxy for analyzing convergence behavior, parallelization opportunities, and hardware efficiency. This provides a practical context for examining how algorithm design scales across different compute architectures.

1.1 Evaluation methods

The error measurement used in this report is the Root mean squared measurement. To compare between original and rendered image, the root mean squared error is defined as

$$e_{rms} = \sqrt{\frac{1}{MN} \sum_x^{M-1} \sum_y^{N-1} [I(x, y) - \tilde{I}(x, y)]^2}. \quad (1)$$

2 CPU

3 GPU

4 Results

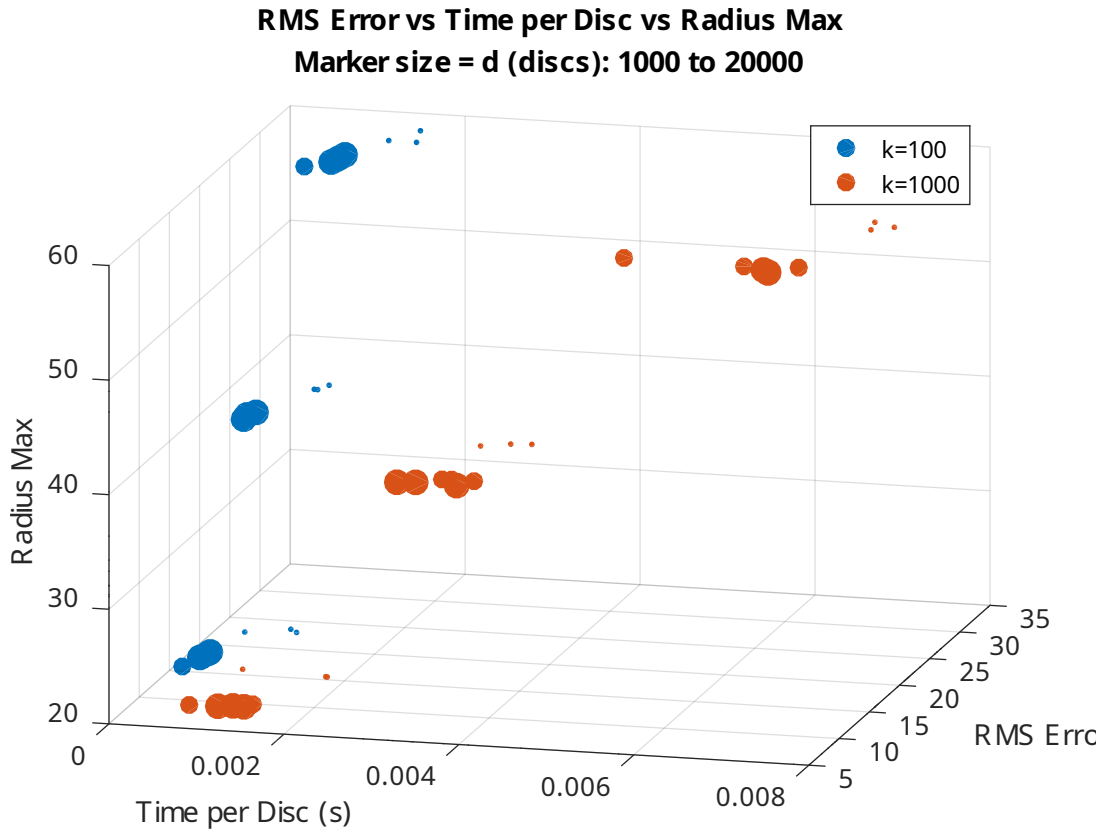


Figure 4.1: Plot

hmmm

5 Conclusions and Discussion

6 Appendix